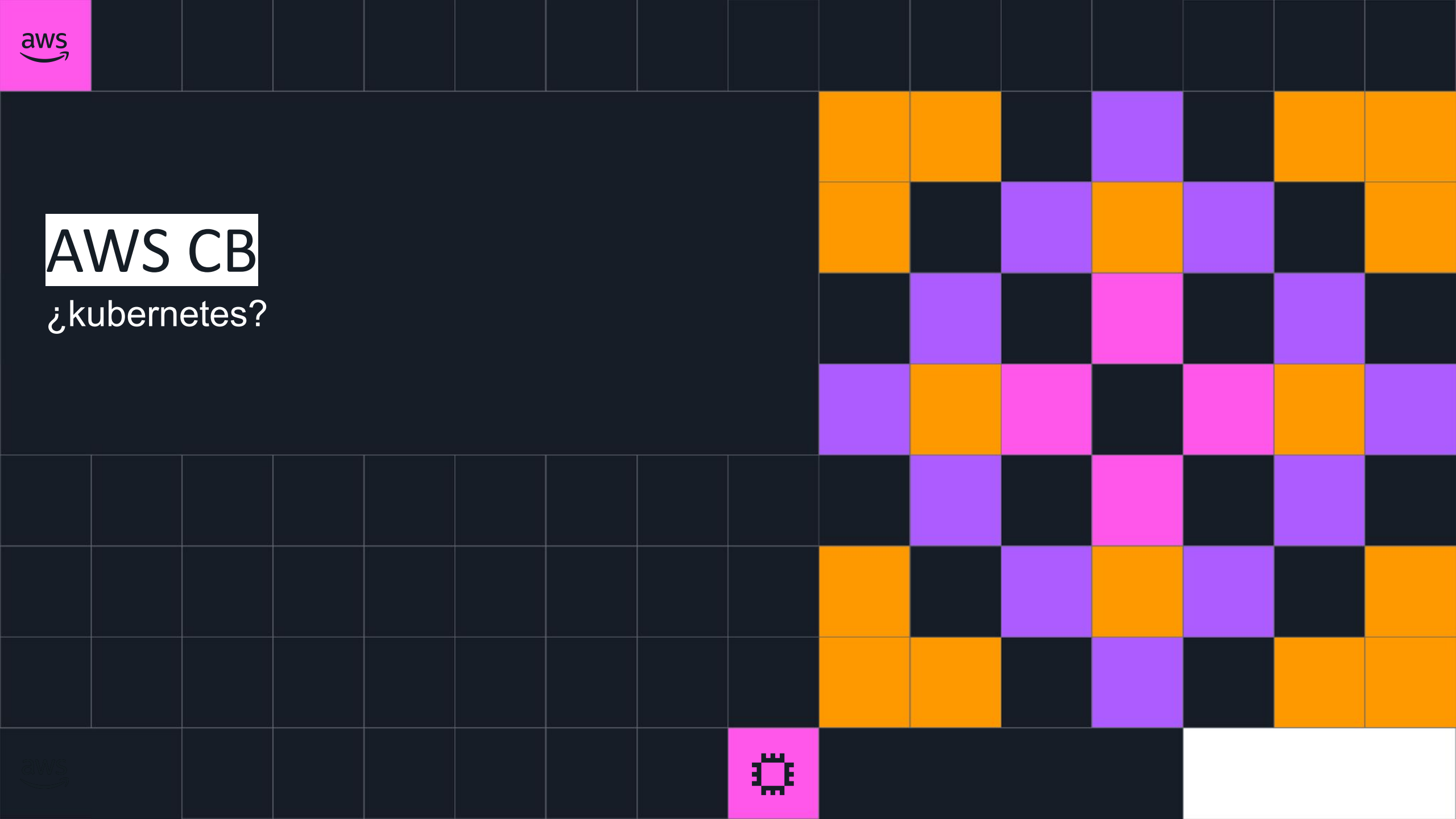
AWS CB

¿kubernetes?



Jhonnatan Gil / JhonnyPong

DevOps / SRE / observability

Globant

CB - Contenedores



/in/jhonnypong/

MÓDULO 1: CONTENEDORES Y K8S

La Evolución del Despliegue de Software



1. Bare Metal

Servidores Físicos Directos

Un solo sistema operativo por máquina. Desperdicio masivo de recursos computacionales e instalación manual propensa a errores.



2. Máquinas Virtuales

Hipervisores e Infraestructura

Múltiples SO Guest sobre un hipervisor. Mayor aislamiento pero alto consumo de RAM y tiempos de arranque lentos (minutos).



3. Contenedores

Aislamiento a Nivel de SO

Kernel compartido. Ligeros, arrancan en milisegundos y empaquetan la aplicación con exactamente sus dependencias.

¿Por qué usar Contenedores?

- **Solución al "En mi máquina sí funciona":** Entorno de ejecución 100% idéntico en desarrollo, staging y producción.
- **Portabilidad Total:** Corre de la misma forma en tu laptop, en servidores locales o en cualquier nube pública.
- **Aislamiento Eficiente:** Cada proceso tiene sus propios recursos limpios sin pagar el costo de una VM completa.



Analogía Comercial

Como un contenedor de carga estandarizado: no importa si va en barco, tren o camión, el contenido y la forma de manipularlo no cambian.

El Límite de Docker en Producción



¿Qué pasa a escala real?

Docker ejecuta contenedores individualmente en un solo nodo. ¿Qué sucede cuando tienes 100 contenedores distribuidos en 10 servidores y uno se cae a las 3 AM?

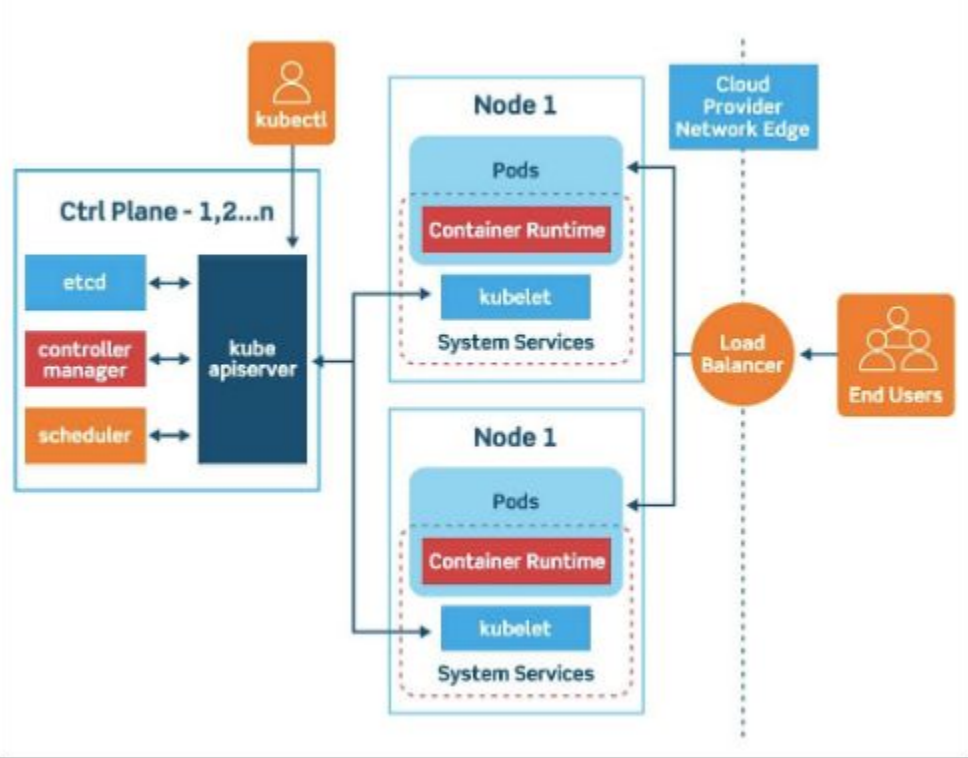
- **¿Quién reinicia los contenedores?** Fallos de memoria o app crash.
- **¿Cómo escala el tráfico?** Pico de usuarios a mediodía.
- **¿Cómo actualizas sin caídas?** Despliegues continuos sin downtime.
- **¿Cómo descubres servicios?** Asignación y balanceo de red dinámico.

¿Qué es Kubernetes (K8s)?

El Orquestador de Contenedores

Plataforma de código abierto para automatizar el despliegue, escalado y la gestión de aplicaciones en contenedores.

- **Origen:** Diseñado por Google basándose en su sistema interno Borg.
- **Comunidad:** Donado a la CNCF (Cloud Native Computing Foundation) en 2015.
- **Abreviación:** "K8s" se debe a las 8 letras entre la K y la s.



MÓDULO 1: CONTENEDORES Y K8S

Tres Pilares Clave de Kubernetes



Auto-Healing

Reinicia contenedores que fallan, reemplaza y reagenda cuando los nodos mueren, y elimina los que no responden a health checks.



Auto-Scaling

Escala automáticamente la cantidad de contenedores arriba o abajo según el uso de CPU, memoria o métricas personalizadas.



Automated Rollouts

Despliega cambios progresivamente inspeccionando la salud. Si algo sale mal, deshace el cambio (rollback) instantáneamente.

MÓDULO 1: CONTENEDORES Y K8S

Arquitectura: El Plano de Control (Control Plane)

kube-apiserver

La puerta de entrada al clúster. Expone la API REST de Kubernetes y procesa todas las peticiones (kubectl/UI).

etcd

Base de datos clave-valor distribuida y consistente. Guarda el estado deseado y real de todo el clúster.

kube-scheduler

Decide en qué Worker Node específico debe ejecutarse un nuevo Pod analizando recursos disponibles.

kube-controller-manager

Ejecuta los bucles de control continuo que comparan el estado actual con el estado deseado.

MÓDULO 1: CONTENEDORES Y K8S

Arquitectura: Los Nodos de Trabajo (Worker Nodes)



kubelet

El agente principal que corre en cada nodo. Se asegura de que los contenedores descritos en los specs se estén ejecutando sanamente.



kube-proxy

Proxy de red que mantiene las reglas de red en los nodos. Permite la comunicación dentro y fuera del clúster.



Container Runtime

El software responsable de ejecutar los contenedores reales (ej. containerd, CRI-O).

K8s Genérico vs. K8s Administrado

🛠️Self-Hosted / DIY

Instalas y mantienes el Control Plane tú mismo.

- Alta complejidad operativa.
- Responsable de actualizar etcd.
- Gestión manual de alta disponibilidad.

🌩️Managed (Nube)

El proveedor administra el Control Plane.

- AWS EKS, Google GKE, Azure AKS.
- Control Plane altamente disponible.
- Te enfocas solo en tus aplicaciones.

AWS EKS (Elastic Kubernetes Service)

Kubernetes nativo en la nube de AWS

- **Control Plane Gestionado:** AWS opera etcd y API Server en múltiples zonas de disponibilidad.
- **Tú manejas los Workers:** Nodos EC2 o Fargate (Serverless).
- **Integración Nativa:** Conexión directa con AWS IAM, VPC, ALB y CloudWatch.



Responsabilidad Compartida

AWS garantiza la disponibilidad de la API de Kubernetes. Tu equipo solo despliega sus Pods y define los requerimientos de cómputo.

MÓDULO 2: PODS

¿Qué es un Pod?

Unidad Básica de K8s

La unidad mínima desplegable

Kubernetes no despliega contenedores directamente; siempre los envuelve en un **Pod**.

- Representa un único proceso de ejecución en tu clúster.
- Puede contener un contenedor principal o un grupo estrechamente acoplado.



Analogía: La Cáscara de Guisante

El Pod es la cáscara (Pod) y los contenedores son los guisantes dentro. Comparten el mismo entorno protegido.

MÓDULO 2: PODS

Anatomía Interna de un Pod

IP Compartida

Todos los contenedores del mismo Pod comparten la dirección IP y puerto (localhost entre ellos).

Volúmenes Compartidos

Pueden montar los mismos volúmenes para intercambiar archivos locales de forma ultrarrápida.

Patrón Sidecar

Un contenedor secundario ayuda al principal (ej. recolector de logs, proxy de telemetría).

¿Por qué NO creamos Pods manualmente?

💀Los Pods son Efímeros

Si creas un Pod directamente y el nodo muere, el Pod NO se recrea. Pierdes la aplicación.

- **Concepto Fundamental:** Estado Deseado (Desired State).
- Tú le dices a K8s: *"Quiero 3 réplicas de mi app corriendo siempre"*.



Bucle de Control Declarativo

K8s monitorea continuamente. Si Estado Actual ≠ Estado Deseado, toma acciones inmediatas para corregirlo automáticamente.

MÓDULO 3: DEPLOYMENTS

ReplicaSets y Deployments

Nivel Alto

Deployment

Objeto de nivel superior. Gestiona las actualizaciones declarativas, los rollbacks y las estrategias de despliegue.

Nivel Medio

ReplicaSet

Garantiza el número exacto de Pods en ejecución en cualquier momento dado.

Ejecución

Pods

Las instancias reales con los contenedores ejecutando el código.

MÓDULO 4: SERVICIOS

El Problema de la Efimeridad de las IPs

⚡Las IPs de los Pods cambian constantemente

Cuando un Pod muere y es reemplazado por un Deployment, el nuevo Pod recibe una IP totalmente distinta.

¿Cómo hace un frontend para comunicarse de forma confiable con un backend si sus IPs dinámicas están cambiando en todo momento?



La Solución: Services

Un **Service** de K8s provee un punto de acceso estable (IP fija y nombre DNS) que balancea el tráfico entre Pods cambiantes.

MÓDULO 4: SERVICIOS

Tipos de Servicios principales

ClusterIP

Por defecto. Expone el servicio en una IP interna del clúster. Solo accesible desde dentro del clúster.

NodePort

Expone el servicio en un puerto estático (30000-32767) de cada nodo. Accesible externamente vía NodoIP:NodePort.

LoadBalancer

Crea un balanceador de carga externo en la nube (ej. AWS ALB/NLB) asignando una IP pública real.

MÓDULO 4: SERVICIOS

Conexión mediante Labels y Selectors

Los Servicios no usan nombres de máquina rígidos. Utilizan un sistema flexible de etiquetas clave-valor.

- **Labels (en Pods):** Etiquetas asignadas como `app: flask-api`.
- **Selector (en Service):** Filtro que indica *"envía tráfico a cualquier Pod que tenga la etiqueta app: flask-api"*.

```
# Manifest del Service
apiVersion: v1
kind: Service
metadata:
  name: backend-svc
spec:
  selector:
    app: flask-api # ⟵ Coincide con la Label del Pod
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
```

MÓDULO 5: CONFIGMAPS

Principio Twelve-Factor App: Configuración

🚫Mala Práctica

Escribir URLs de base de datos o flags de entorno quemados (hardcoded) dentro de la imagen del contenedor.

✅Buena Práctica (12-Factor)

Estricta separación del código fuente y la configuración. La misma imagen corre en Dev, QA y Prod.



Inyección Externa

La aplicación debe leer su configuración desde el entorno de ejecución, permitiendo cambiar el comportamiento sin recompilar.

MÓDULO 5: CONFIGMAPS

Uso de ConfigMaps en Kubernetes

Un **ConfigMap** guarda información no confidencial en pares clave-valor.

Dos formas de inyección:

- **Variables de entorno:** Directamente legibles por el proceso de la app.
- **Archivos montados:** Como archivos en un volumen dentro del Pod (ej. `/etc/config/app.json`).

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  DB_HOST: "postgres-service"
  PORT: "8080"
  LOG_LEVEL: "debug"
```

MÓDULO 6: SECRETS

Manejo Seguro: ConfigMaps vs. Secrets

ConfigMap

Datos Públicos / Texto Plano

Ideal para puertos, niveles de log, variables no sensibles. Visible directamente en YAMLs.

Secret

Datos Sensibles

Para contraseñas, API keys, tokens JWT y certificados SSL. Almacenamiento protegido e inyección segura.

MÓDULO 6: SECRETS

Almacenamiento y Uso de Secrets

⚠ Importante: Base64 NO es encriptación

Los valores en los manifiestos de Secrets están codificados en Base64 solo para soportar binarios. Deben protegerse en reposo (etcd encryption) e IAM.

- Montaje como archivos tmpfs (memoria RAM) para que nunca toquen el disco.

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
data:
  # 'admin' codificado en base64
  username: YWRtaW4=
  # 'supersecret' codificado
  password: c3VwZXJzZWNyZXQ=
```

MÓDULO 7: INGRESS

El Reto del Enrutamiento HTTP/HTTPS

Por qué NodePort / LoadBalancer no bastan:

- **Costo Elevado:** Un Service LoadBalancer crea un balanceador de nube pagado por cada app.
- **Sin Capa 7 (Aplicación):** No pueden enrutarse peticiones por ruta HTTP (/api vs /web).
- **Gestión de TLS/SSL:** Complejo de administrar en cada servicio individual.



Necesitamos Capa 7

Un único punto de entrada inteligente que inspeccione URL y Headers antes de redirigir.

MÓDULO 7: INGRESS

¿Qué es Ingress?



Enrutamiento por Dominio

Dirige api.miempresa.com al backend y app.miempresa.com al frontend.



Enrutamiento por Ruta

Dirige miempresa.com/users a un servicio y miempresa.com/orders a otro.



Terminación SSL/TLS

Centraliza el certificado HTTPS en la entrada sin recargar los servicios internos.

MÓDULO 7: INGRESS

Ingress Resource vs. Controller

La Definición

Ingress Resource (YAML)

Archivo de especificación donde declaras las reglas de tráfico. Solo es una "lista de deseos", no ejecuta nada por sí solo.

El Motor

Ingress Controller

El programa real (ej. NGINX, Traefik, AWS ALB Controller) que lee los objetos Ingress y configura el balanceador activo.

MÓDULO 8: GATEWAY API

La Evolución: Kubernetes Gateway API

¿Por qué nació esta nueva especificación?

Ingress se diseñó en 2015 y era demasiado monolítico. Requería muchas annotations no estandarizadas para casos avanzados.

- **Estandarización:** Expresividad nativa sin annotations raras.
- **Orientación por Roles:** Separa responsabilidades entre equipos.



Siguiente Generación

Gateway API es el futuro oficial de la red en Kubernetes, diseñado para arquitectura de microservicios multiequipo.

MÓDULO 8: GATEWAY API

Orientación por Roles en Gateway API



Infra Provider

Define la GatewayClass (ej. controlador de AWS, Istio o NGINX instalado en el clúster).



Cluster Admin

Crea el recurso Gateway, configurando puertos, IP pública y certificados SSL globalmente.

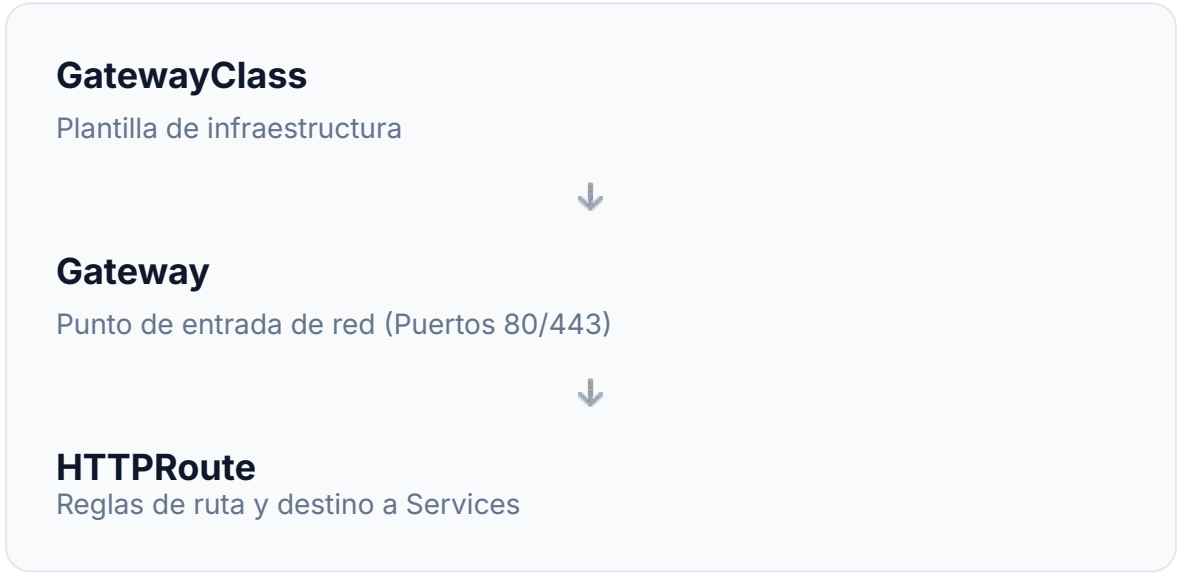


Developer

Escribe recursos HTTPRoute adjuntándolos a la Gateway para definir rutas de su app.

MÓDULO 8: GATEWAY API

Arquitectura de Recursos en Gateway API



- **Desacoplamiento:** Los desarrolladores no necesitan pedir permisos de red para agregar una nueva ruta HTTP.
- **Soporte Multi-protocolo:** Soporta nativamente HTTP, HTTPS, gRPC, TCP y UDP.

Dockerfile FROM python:3.11-slim WORKDIR /app RUN pip install flask COPY app.py . EXPOSE 5000 CMD ["python", "app.py"]

MÓDULO 9: DEMO PRÁCTICA

La Aplicación de Prueba (Python Flask)

```python
# app.py
from flask import Flask
import os

app = Flask(__name__)

@app.route('/')
def hello():
    pod_name = os.getenv('POD_NAME', 'unknown')
    return f"Hola desde K8s! Pod: {pod_name}\n"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

```dockerfile
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
RUN pip install flask
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

https://killercoda.com/course-ckad/scenario/introduction

https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

MÓDULO 9: DEMO PRÁCTICA

El Entorno Local con `kind`

Kubernetes in Docker

`kind` corre nodos de K8s como si fueran contenedores Docker en tu laptop. Ideal para desarrollo local rápido.

- **Crear Clúster:** Comando ultrarrápido en 1 minuto.
- **Carga de Imagen:** Permite subir imágenes locales sin publicarlas a DockerHub.

```
# 1. Crear el clúster local
kind create cluster --name demo-k8s

# 2. Construir la imagen Docker
docker build -t flask-demo:v1 .

# 3. Cargar la imagen directamente a kind
kind load docker-image flask-demo:v1 --name demo-k8s
```

service.yaml apiVersion: v1 kind: Service metadata: name: flask-service spec: type: NodePort selector: app: flask-app ports: - port: 80 targetPort: 5000 nodePort: 30080

MÓDULO 9: DEMO PRÁCTICA

Los Manifiestos YAML (Deployment & Service)

```yaml
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: flask-app
  template:
    metadata:
      labels:
        app: flask-app
    spec:
      containers:
      - name: flask
        image: flask-demo:v1
        ports:
        - containerPort: 5000
```

```yaml
# service.yaml
apiVersion: v1
kind: Service
metadata:
  name: flask-service
spec:
  type: NodePort
  selector:
    app: flask-app
  ports:
    - port: 80
      targetPort: 5000
      nodePort: 30080
```

MÓDULO 9: DEMO PRÁCTICA

Despliegue y Verificación en Local

```
# Aplicar los manifiestos
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml

# Verificar recursos
kubectl get pods
# NAME                                READY   STATUS
# flask-deployment-687b99c7b8-x29lp   1/1     Running
# flask-deployment-687b99c7b8-z8m2q   1/1     Running

kubectl get svc flask-service

# Probar la aplicación vía Port Forward
kubectl port-forward svc/flask-service 8080:80
curl http://localhost:8080
```



¡Felicidades!

Has desplegado tu primera aplicación en contenedores orquestada localmente con Kubernetes y kind.

Premios?

https://forms.gle/b3GQgjxcwFyYYeEk6



MUCHAS GRACIAS

Jhonnatan Gil / JhonnyPong

DevOps / SRE / observability

Globant

CB - Contenedores



/in/jhonnypong/